

Bob Break

A Developer Attention Management System for Agentic AI

While Bob Works, You Breathe

The Problem

AI development agents can now generate code, analyze documentation, execute tests, and coordinate multiple tasks faster than a human developer can comfortably follow.

This creates a new bottleneck:

The bottleneck is no longer how fast AI can generate code. It is how much information a human developer can process without becoming overwhelmed.

Developers already spend hours looking at visually dense information: source code, terminal output, logs, test results, documentation, pull requests, and error messages.

Tools such as IBM Bob reduce the amount of manual work developers need to perform. However, when several agents work simultaneously, they can also produce a continuous stream of plans, updates, questions, code changes, and results.

The developer may feel compelled to monitor everything:

- Is Bob still working?
- Is one of the agents blocked?
- Has an important error appeared?
- Does Bob need a decision?
- Should I read every update?
- Can I safely look away?

Even while Bob is doing the work, the developer's eyes and brain rarely get a genuine break.

This results in:

- Visual fatigue from continuously reading code and technical text.
- Cognitive overload from following several agents simultaneously.
- Repeated checking and unnecessary supervision.
- Interruptions to processes that were still progressing correctly.

- Context switching to social media or unrelated applications.
- Important decisions becoming lost among low-priority updates.
- Difficulty understanding the final result after receiving too much information.

AI saves development time, but that saved time can become anxious waiting.

The Solution

Bob Break is a developer attention management system for IBM Bob 2.0.

It transforms autonomous AI processing time into genuine visual recovery while keeping the developer informed and in control.

When Bob and its subagents begin working, Bob Break analyzes their activity and separates information into four categories:

1. **Background progress**
Routine updates are stored for the final summary and represented through calm visual progress.
2. **Developer decision required**
Bob Break pauses the recovery experience and presents one short, clearly structured question.
3. **Blocked task**
The developer receives a visible but non-alarming notification explaining what is preventing progress.
4. **Critical risk**
Bob Break immediately alerts the developer when human attention is genuinely necessary.

Instead of displaying every intermediate message, Bob Break communicates the state of the work through a quiet visual interface.

Detailed logs, agent conversations, and code changes remain available, but they do not continuously occupy the main screen.

Bob reduces the workload. Bob Break reduces the cognitive load.

The Visual Recovery Experience

While Bob works, the developer can choose a low-stimulation experience.

Growing Garden

Each Bob subagent is represented by a plant.

As an agent progresses, its plant grows. When the task is completed, it flowers or releases fireflies. If an agent becomes blocked, its plant gently changes state.

The garden provides a readable overview of the workflow without requiring the developer to follow continuous technical text.

Breathing Mode

A soft circle expands and contracts to guide the developer through a 30–60 second breathing or eye-rest exercise.

Color changes communicate overall progress without showing logs or code.

Companion Mode

A small Tamagotchi-style companion rests alongside the developer.

The companion breathes, stretches, waters the garden, or sleeps while Bob works. Its behavior reflects the overall state of the development process.

Bubble Mode

The developer can slowly pop floating bubbles during a short break. The interaction is deliberately simple and does not become another demanding source of stimulation.

Bob Break is not designed to capture the developer's attention.

It is designed to release it.

Prototype Workflow

The working prototype will demonstrate Bob Break on a sample application.

The developer gives IBM Bob the following request:

“Add dark mode to the application, review accessibility, create automated tests, and update the project documentation.”

Step 1: Planning

IBM Bob uses Agent mode to inspect the request, understand the sample project, and create an implementation plan.

Step 2: Document Understanding

Bob reviews the project requirements, architecture notes, accessibility guidelines, coding conventions, and existing documentation.

This ensures that the work follows the project's actual constraints instead of treating the request as an isolated coding task.

Step 3: Parallel Execution

Bob assigns the work to specialized subagents:

- **UI subagent:** Implements dark mode.
- **Accessibility subagent:** Reviews contrast, keyboard navigation, labels, and semantic structure.
- **Testing subagent:** Creates and executes automated tests.
- **Documentation subagent:** Updates the README and release notes.

These tasks run in parallel.

Step 4: Bob Break Activation

As soon as the autonomous work begins, Bob Break switches the interface from technical monitoring to visual recovery mode.

Each subagent becomes a plant, firefly, or visual element in the garden.

The developer can understand the overall state at a glance without reading every update.

Step 5: Attention Management

Bob Break processes incoming activity:

- Routine updates are grouped silently.
- Progress is translated into visual changes.
- Questions are reduced to a clear decision card.
- Blocked tasks generate a concise notification.
- Critical problems trigger an immediate alert.

The developer is interrupted only when human input provides real value.

Step 6: Completion and Return

When all agents finish, Bob Break uses a gentle transition to bring the developer back to the technical workspace.

Instead of displaying a wall of generated text, it presents a progressive summary:

- What Bob changed.
- Which agents completed their tasks.
- Which tests passed or failed.
- Which documentation was updated.
- Which decisions Bob made.
- Which risks remain.
- Which files require human review.
- What the developer should do next.

The developer can then choose whether to inspect the code, review the tests, or continue with the next task.

Prototype Interface

The MVP contains three areas:

Agent Activity

A simplified list of the active subagents and their states:

- Planned.
- Working.
- Waiting.
- Blocked.
- Completed.

Visual Recovery

The central space displays the garden, companion, breathing animation, or bubble experience.

It deliberately avoids dense technical information.

Essential Progress

A minimal overview shows:

- Overall progress.
- Active and completed agents.
- Whether developer input is needed.
- Important blockers.
- Current development phase.

A “View technical details” option remains available, but it is never forced upon the developer.

Why This Is More Than a Loading Screen

A loading screen only indicates that something is happening.

Bob Break actively manages the relationship between AI activity and human attention.

It:

- Understands different types of agent events.
- Filters low-value updates.
- Identifies when human intervention is required.
- Combines several parallel agents into one understandable state.
- Protects the developer from unnecessary information.
- Preserves access to full technical details.
- Creates a structured return to the code.
- Produces a concise final summary.

Bob Break is not a distraction from development. It is part of the development workflow.

Expected Impact

The prototype aims to demonstrate that Bob Break can:

- Reduce repeated checking of AI progress.
- Reduce unnecessary context switching.
- Reduce continuous exposure to code, logs, and technical text.
- Reduce interruptions to autonomous agent processes.
- Make blockers and decisions easier to identify.
- Help developers understand parallel agent activity at a glance.
- Turn waiting time into a short visual and cognitive recovery period.
- Make the final result easier to review.
- Help developers return to their work with greater focus.

Measuring the Impact

The demonstration will compare the same development task with and without Bob Break.

We will measure:

- Number of times the developer checks agent progress.
- Number of context switches.
- Time spent reading intermediate technical output.
- Number of unnecessary interruptions.
- Time required to understand the final result.

- Number of important decisions or warnings missed.
- Developer-reported visual and cognitive load.

Metric	Traditional agent workflow	Bob Break target
Progress checks	Frequent	Only when necessary
Context switches	4–6	0–1
Intermediate output reviewed	Almost everything	Only actionable information
Continuous technical-text exposure	High	Significantly reduced
Intentional visual recovery	None	30–90 seconds
Visibility across parallel agents	Fragmented	Unified
Final review	Unstructured output	Progressive summary

These are prototype targets that will be validated during the demonstration rather than presented as existing results.

Why IBM Bob 2.0?

Bob Break is made possible by IBM Bob’s ability to:

- Plan complex development work.
- Understand project documentation.
- Coordinate agents and subagents.
- Execute independent tasks in parallel.
- Validate code and tests.
- Manage a multi-step development workflow.
- Summarize completed work.

Bob’s autonomy creates periods in which constant human supervision is unnecessary.

Bob Break makes those periods visible, understandable, and restorative.

The visual experience is connected to real agent activity, task progress, decisions, blockers, and results. It is therefore not a separate wellness application placed next to Bob. It is an attention-management layer built around Bob’s agentic workflow.

Why Bob Break Matters

Most AI development tools are designed to increase the speed and volume of generated work.

Bob Break addresses the human limit behind that acceleration.

As AI agents become faster and more autonomous, developers will not be able—or need—to follow every generated line, message, and intermediate decision.

The next generation of development tools must manage not only tasks and code, but also human attention.

Bob Break explores that future.

It helps IBM Bob communicate at a human pace, allowing developers to remain informed without becoming overwhelmed.

Conclusion

Bob Break transforms one of the most neglected moments in AI-assisted development: the time between assigning a complex task and receiving its result.

IBM Bob plans, understands, delegates, executes, tests, and documents the work.

Bob Break decides what deserves the developer's attention and what can wait.

Together, they create a workflow that is not only faster, but calmer, clearer, and more sustainable.

AI should save developers time—not turn that time into anxious waiting.

Bob reduces the workload. Bob Break reduces the cognitive load.

While Bob works, you breathe.